Plataformas de Desenvolvimento

Ricardo Filipe (ricardo.filipe@isec.pt)

Mestrado em Informática e Sistemas

Docker

- Why containers?
- What are containers and images?
- Containers basic commands
- Containers networks
- Image concepts and creation
- Volumes
- Containers Registry

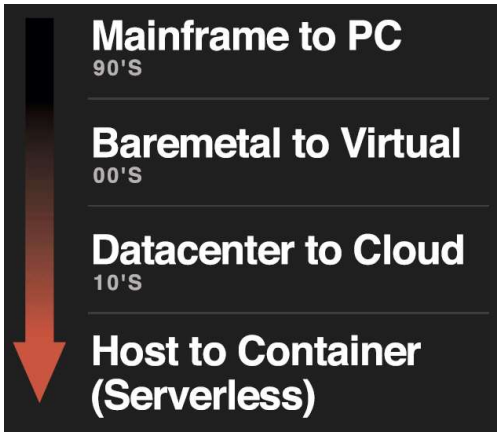
Docker

- Other topics (just some insights)
  - Docker compose
  - Docker Swarm
  - Kubernetes

Docker



Docker - evolution



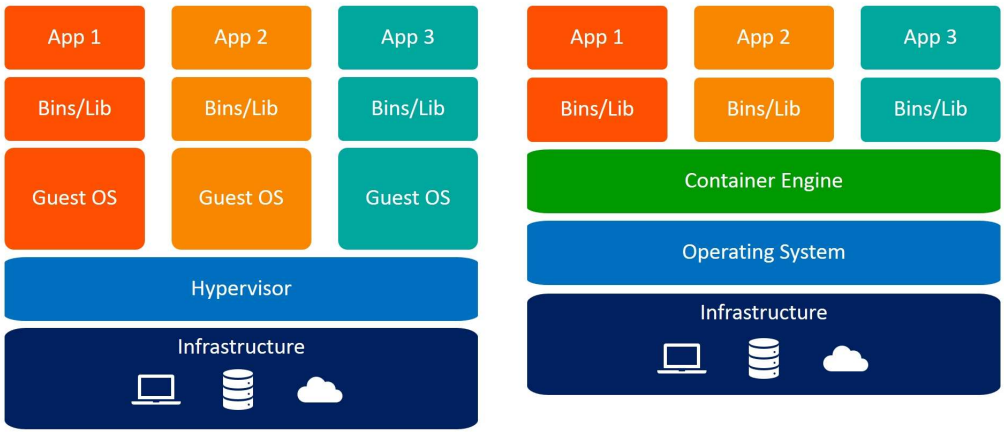
5

Docker containers

- Two type of containers: Linux Containers and Windows Containers
- Possible to install on Windows, MAC or Linux

6

Containers versus VMs



Virtual Machines Containers

7

Containers versus VMs

Container Pros:

•Containers are more lightweight than VMs, as their images are measured in megabytes rather than gigabytes
•Containers require fewer IT resources to deploy, run, and manage
•Containers spin up in milliseconds since their order of magnitude is smaller
•A single system can host many more containers as compared to VMs

Container Cons:
•All containers must run atop the same OS – no mix and match of OSs or versions
•Containers may be less secure than VMs since the underlying OS is shared
•Containers are a newer technology, and the ecosystem is still evolving

https://www.vmware.com/topics/glossary/content/vms-vs-containers.html

8

Containers versus VMs

- Containers aren’t Mini-VM’s
- They are just processes
- Limited to what resources they can access (file paths, network devices, running processes)
- Exit when process stops
- docker run --name nginx -d nginx
- docker top nginx
- ps aux
- docker stop nginx
- ps aux

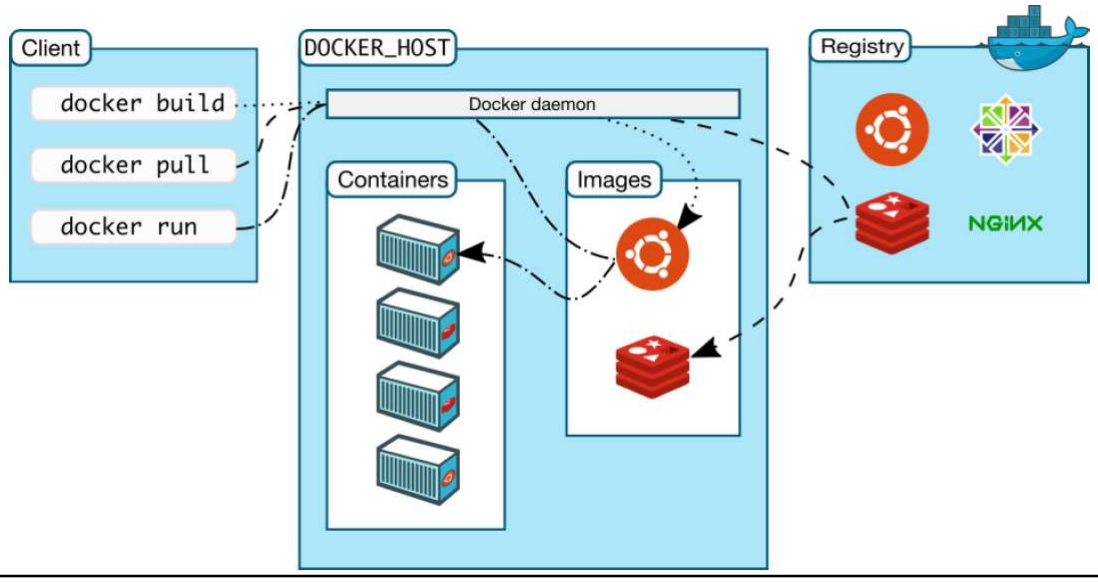
9

Containers versus Images

- An Image is the application we want to run
- A Container is an instance of that image running as a process
- Might have several containers running for the same image.
- Our default image will be the Nginx web server
- Docker's default image "registry" is called Docker Hub (hub.docker.com)
- You might have other custom registries.

10

Docker – Overall flow



11

Docker basic commands

- docker container run --publish 80:80 nginx

- Steps executed by docker:

1. Download nginx image (latest).
2. Start a new container from that image.
3. Open port 80 on the HOST IP
4. Routes traffic from port 80 (host) to port 80 (container)

12

Docker basic commands

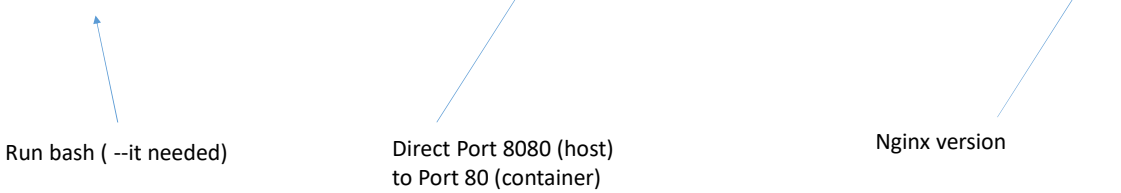
- docker container run --publish 80:80 nginx
- 1. Looks for that image locally in image cache, doesn't find anything
- 2. Then looks in remote image repository (defaults to Docker Hub)
- 3. Downloads the latest version (nginx:latest by default)
- 4. Creates new container based on that image and prepares to start
- 5. Gives it a virtual IP on a private network inside docker engine
- 6. Opens up port 80 on host and forwards to port 80 in container
- 7. Starts container by using the CMD in the image Dockerfile

13

Docker basic commands

- Changing defaults

docker container run -it --publish **8080:80** --name webhost -d nginx**:1.11** **bash**



14

Docker basic commands

- docker container top - process list in one container
- docker container inspect - details of one container config
- docker container stats - performance stats for all containers
- docker container run -it - start new container interactively
- docker container exec -it - run additional command in existing container

15

Docker Networks

- Review of docker container run -publish
- For local dev/testing, default networks usually "just work"
- Quick port check with docker container port <container>
- Learn concepts of Docker Networking
- Understand how network packets move around Docker

16

Docker Networks

- Each container connect to a private virtual network "bridge"
- Each virtual network routes through NAT firewall on host IP
- All containers on a virtual network can talk to each other without -p
- Best practice is to create a new virtual network for each applicational function:
- network "my_web_app" for mysql and php/apache containers
- network "my_api" for mongo and nodejs containers

17

Docker Networks

- Make new virtual networks
- Attach containers to more then one virtual network (or none)
- Skip virtual networks and use host IP (--net=host)

18

Docker Networks

- Show networks: docker network ls
- Inspect a network: docker network inspect
- Create a network: docker network create --driver
- Attach a network to container: docker network connect
- Detach a network from a container: docker network disconnect

19

Docker Networks

- Create your apps so frontend/backend sit on same Docker network
- Their inter-communication never leaves host
- All externally exposed ports closed by default
- You must manually expose via -p, which is better default security!
- This gets even better later with Swarm and Overlay networks

20

Docker Networks

- Containers shouldn't rely on IP's for inter-communication
- DNS for friendly names is built-in if you use custom networks

Docker Images

- Official definition: "An Image is an ordered collection of root filesystem changes and the corresponding execution parameters for use within a container runtime."
- Not a complete OS. No kernel, kernel modules (e.g. drivers)
- Small as one file (your app binary) like a golang static binary
- Big as an Ubuntu distro with apt, and Apache, PHP, and more installed

Docker Images

- Created Using a **Dockerfile**
- Or committing a containers changes back to an image
- Stored in Docker Engine image cache
- Move images in/out of cache via:
    - local filesystem via tarballs
    - push/pull to a remote "image registry" (e.g. Docker Hub)
- Images aren't ideal for persistent data
    - Mount a host file system path into container
    - Use **docker volume** to create storage for unique/persistent data

Docker Images

- Images are made up of file system changes and metadata
- Each layer is uniquely identified and only stored once on a host
- This saves storage space on host and transfer time on push/pull
- A container is just a single read/write layer on top of image
- docker image history and inspect commands can teach us

Docker Images

- Dockerfile basics

- ARG (build argument)
- ADD (add file to container)
- ENV (environment variable)
- RUN (any arbitrary shell command)
- EXPOSE (informative - open port from container to virtual network)
- CMD (command to run when container starts)
- ENTRYPOINT (default script to execute upon container run)
- docker image build (create image from Dockerfile)

25

Docker Images

```
FROM ubuntu:18.04
COPY . /app
RUN make /app
CMD python /app/app.py
```

Each instruction creates one layer:

- `FROM` creates a layer from the `ubuntu:18.04` Docker image.
- `COPY` adds files from your Docker client's current directory.
- `RUN` builds your application with `make` .
- `CMD` specifies what command to run within the container.

26

Docker Images

https://phoenixnap.com/kb/docker-cmd-vs-entrypoint

27

Docker Images

docker image tag nginx-student:latest ricardofilipe/nginx-student:latest

docker push ricardofilipe/nginx-student

- Tag image with a specific tag
- Send to docker hub repository (free)



28

Docker Volumes

- Containers are **usually** immutable and ephemeral
- "immutable infrastructure": only re-deploy containers, never change
- This is the ideal scenario, but what about databases, or unique data?
- Docker gives us features to ensure these "separation of concerns"
- This is known as "persistent data"
- Two ways: Volumes and Bind Mounts
- Volumes: make special location outside of container UFS
- Bind Mounts: link container path to host path

Docker Volumes

- **VOLUME** command in **Dockerfile**
- Also override with **docker run -v /path/in/container**
- Bypasses Union File System and stores in alt location on host
- Includes it's own management commands under **docker volume**
- Connect to none, one, or multiple containers at once
- Not subject to **commit, save, or export** commands
- By default they only have a unique ID, but you can assign name
- Then it's a "named volume"

Docker Volumes

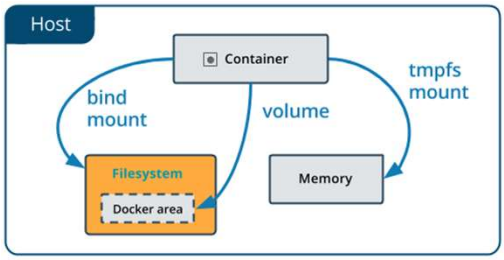
- **Bind Mounting**

Maps a host file or directory to a container file or directory

- Basically just two locations pointing to the same file(s)
- Again, skips UFS, and host files overwrite any in container
- Can't use in Dockerfile, must be at container run
- ... run -v /Users/stuff:/path/container (mac/linux)

31

Docker: Bind versus Volume



https://docs.docker.com/storage/bind-mounts/

32

Docker Best Practices

- **Minimize number of layers**

Same goal: Unzip file.
Image on the left with 3 layers.
Image on the right with 1 layer.
Image on the left will have the zip (i.e. the full sized file) on the first layer.

Cons:
Any change to the command on the right will create a new layer (normally avoided by the cache docker instruction)

```
COPY <filename>.zip <copy_directory>
RUN unzip <filename>.zip
RUN rm <filename>.zip
```

```
RUN curl <file_download_url> -O <copy_directory> \
&& unzip <copy_directory>/<filename>.zip -d <copy_directory> \
&& rm <copy_directory>/<filename>.zip
```


Docker Best Practices

- **Minimize size of layers**

Clean all data that is not mandatory to the image – e.g. yum clean all

```
RUN yum install -y epel-release && \
    rpmkeys --import file:///etc/pki/rpm-gpg/RPM-GPG-KEY-CentOS-7 && \
    yum install -y --setopt=tsflags=nodocs bind-utils gettext iproute\
    v8314 mongodb24-mongodb mongodb24 && \
    yum -y clean all
```

Docker Best Practices

- **Use (and reuse build arguments)**

Avoid copy paste for distinct versions of the same SW. Use arguments to reuse dockerfiles.

https://docs.docker.com/engine/reference/builder/#arg

```
ARG my_version=1.2.3
ARG my_sha256=cafebabedeadbeef...
RUN set -e ;\
    curl -o thing-${my_version} https://example.com/thing/${my_version}/thing-${my_version}.tar.gz ;\
    printf '%s  %s' "${my_sha256}" "thing-${my_version}" | shasum -a 256 -c ;\
    echo everything checks out
```

35

Docker Best Practices

- **Try not to use root privileges**

```
RUN groupadd -r myapp && useradd -r -g myapp myapp
USER myapp
```

Running containers as root is a bad idea for security. This has been shown time and time again. Hackers find new ways of escaping out of the container, and that grants unfettered access to the host or Kubernetes node

https://blog.aquasec.com/cve-2022-0185-linux-kernel-container-escape-in-kubernetes

36

Docker Best Practices

- **Multi-stage builds to smaller images**

Multistage builds are useful to anyone who has struggled to optimize Dockerfiles while keeping them easy to read and maintain.

Use a previous stage as a new stage

You can pick up where a previous stage left off by referring to it when using the `FROM` directive. For example:

```
FROM alpine:latest as builder
RUN apk --no-cache add build-base

FROM builder as build1
COPY source1.cpp source.cpp
RUN g++ -o /binary source.cpp

FROM builder as build2
COPY source2.cpp source.cpp
RUN g++ -o /binary source.cpp
```


Docker Best Practices

- **Avoid running multiple services/processes in the same container (e.g. using supervisor)**
- 1 container == 1 process

Docker Best Practices

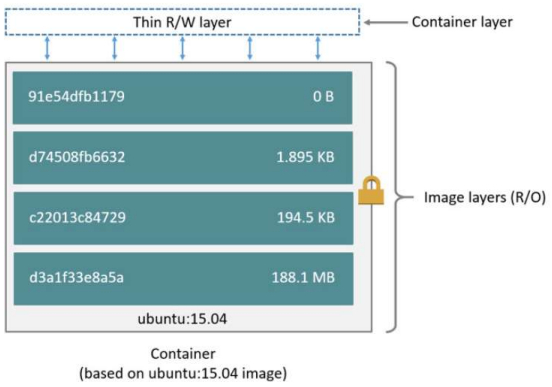
- **Label your dockerfile**

```
LABEL com.example.name="testing" \
      com.example.version="${VERSION}" \
      com.example.release="1" \
      com.example.group="prod" \
      com.example.summary="test" \
      com.example.license="Genral License Terms and Conditions (see http://www.example.com/licenses/)" \
      com.example.packager="testing" \
      com.example.os="ubuntu20"
```


Docker Best Practices

- **Don't store data or logs inside container:**
- Containers must be immutable and ephemeral

Each layer is only a set of differences from the layer before it. The layers are stacked on top of each other. When you create a new container, you add a new writable layer on top of the underlying layers. This layer is often called the "container layer". All changes made to the running container, such as writing new files, modifying existing files, and deleting files, are written to this thin writable container layer. The diagram below shows a container based on the Ubuntu 15.04 image.



Docker Compose

To:

- Configure relantionships between containers
- Save the run settings in a easy-to-read file
- Easy to create development and test environments (not production)

- Agregate 2 things:
  - YAML file describing:
    - Containers
    - Networks
    - Volumes
- CLI tool **docker-compose** to manage those dev/test environments

Docker Compose

docker-compose.yml

Used with docker-compose command or directly in production with docker Swarn (v1.13)

Docker Compose

Simple example:

```
version: "2.4"
services:
  web:
    build: .
    ports:
     - "5000:5000"
    volumes:
     - .:/code
  redis:
    image: redis
```


Docker Compose

A more extended example, defining volumes and networks:

```
version: "2.4"
services:
  web:
    build: .
    ports:
     - "5000:5000"
    volumes:
     - .:/code
    networks:
      - front-tier
      - back-tier
  redis:
    image: redis
    volumes:
      - redis-data:/var/lib/redis
    networks:
      - back-tier
volumes:
  redis-data:
    driver: local
networks:
  front-tier:
    driver: bridge
  back-tier:
    driver: bridge
```

Docker Compose

```
version: "2.4"
services:
  web:
    build: .
    depends_on:
      - db
      - redis
  redis:
    image: redis
  db:
    image: postgres
```


Some bibliography

- https://landscape.cncf.io/
- https://github.com/moby/moby/blob/master/image/spec/v1.md - Image spec
- https://docs.docker.com/storage/ - storage
- https://www.oreilly.com/radar/an-introduction-to-immutable-infrastructure/
- https://www.weave.works/blog/a-practical-guide-to-choosing-between-docker-containers-and-vms
- https://phoenixnap.com/kb/docker-cmd-vs-entrypoint
- https://docs.docker.com/storage/bind-mounts/
- https://docs.docker.com/develop/develop-images/dockerfile_best-practices/
- https://github.com/FuriKuri/docker-best-practices
- https://docs.docker.com/storage/storagedriver/
- https://betterprogramming.pub/docker-best-practices-and-anti-patterns-e7cbccba4f19
- https://docs.docker.com/develop/develop-images/multistage-build/
- https://docs.docker.com/compose/compose-file/compose-versioning/